MLP Coursework

Problem 1

Q1. Fill in all the weights in this network.

The given network consists of 3 input nodes (X1, X2, X3), 3 hidden nodes (H1, H2, H3) and 1 output node (OP).

Input Layer to Hidden Layer:

- H1: W11, W21, W31 (Bias: W01)
- H2: W12, W22, W32 (Bias: W02)
- H3: W13, W23, W33 (Bias: W03)

Hidden Layer to Output Layer:

- H1 to OP: W1op
- H2 to OP: W2op
- H3 to OP: W3op
- Output Bias: W0op

Q2. How many parameters do we have in the network?

To calculate the total parameters, we need to count all weights and biases.

- Input to Hidden Layer: (3 * 3) + 3 biases = 12
- Hidden to Output Layer: 3 weights + 1 bias = 4

Total Parameters = 12 + 4 = 16

Therefore, the network has **16 trainable parameters**.

Q3. Write the forward transformation equation for all the nodes in the hidden layer and output layer.

Since all activation functions are sigmoid, we use the sigmoid function for every hidden and output node.

Hidden Layer:

- H1 = sigmoid(W01 + W11*X1* + *W21*X2 + W31*X3)
- H2 = sigmoid(W02 + W12*X1* + *W22*X2 + W32*X3)
- H3 = sigmoid(W03 + W13*X1* + *W23*X2 + W33*X3)

Output Layer:

OP = sigmoid(W0op + W1op*H1* + *W2op*H2 + W3op*H3)

The final predicted output is represented as Y_hat = OP.

Q4. Compute the gradient for OP weights and H3 weights, assuming the loss function to be MSE.

MSE (Mean Squared Error) is used to calculate the difference between the actual output and predicted output.

For a single training example:

Loss = (Predicted Output - Actual Output)^2

Where:

- Y_hat = Predicted output
- y = Actual output

1. Gradients for Output Weights

Using the chain rule, the gradients are:

- Gradient for W1op:
  2 * (Y_hat - y) * Y_hat * (1 - Y_hat) * H1
- Gradient for W2op:
  2 * (Y_hat - y) * Y_hat * (1 - Y_hat) * H2
- Gradient for W3op:
  2 * (Y_hat - y) * Y_hat * (1 - Y_hat) * H3
- Gradient for W0op (Bias):
  2 * (Y_hat - y) * Y_hat * (1 - Y_hat)

2. Gradients for H3 Weights

Using the chain rule again:

- Gradient for W13:
  2 * (Y_hat - y) * Y_hat * (1 - Y_hat) * W3op * H3 * (1 - H3) * X1
- Gradient for W23:
  2 * (Y_hat - y) * Y_hat * (1 - Y_hat) * W3op * H3 * (1 - H3) * X2
- Gradient for W33:
  2 * (Y_hat - y) * Y_hat * (1 - Y_hat) * W3op * H3 * (1 - H3) * X3
- Gradient for W03 (Bias):
  2 * (Y_hat - y) * Y_hat * (1 - Y_hat) * W3op * H3 * (1 - H3)

These gradients are used during backpropagation to update the weights and reduce the prediction error.

Problem 2

Q1. Is this ANN suitable for multiclass classification? Explain your answer.

No, the given ANN is not directly suitable for multiclass classification because it has only one output neuron with a sigmoid activation function.

- A single sigmoid output is generally used for binary classification.
- Multiclass classification requires multiple output neurons, one for each class.
- Softmax activation can be used to calculate the probability of each class.

For example, if we want to classify Cat, Dog and Bird, we need three output neurons.

Therefore, the given network needs some changes to work for multiclass classification.

Q2. How will you change your formula of 1.4 if we have 2 nodes in the output layer?

If we have two output nodes instead of one, each node will have its own weights and bias.

Output Node 1:

OP1 = sigmoid(W01op + W11op*H1* + *W21op*H2 + W31op*H3)

Output Node 2:

OP2 = sigmoid(W02op + W12op*H1* + *W22op*H2 + W32op*H3)

Both nodes receive inputs from the same hidden layer but calculate their outputs separately.

Q3. How will you change your formula of 1.4 if instead of MSE we use Binary Cross-Entropy loss?

Binary Cross-Entropy (BCE) is commonly used for binary classification. It measures the difference between the actual class and the predicted probability.

1. Binary Cross-Entropy Loss Function

Loss = -[y * log(Y_hat) + (1-y) * log(1-Y_hat)]

Where:

- y = Actual output
- Y_hat = Predicted output

2. Gradients for Output Weights

When BCE is used with sigmoid activation, the gradient calculation becomes simpler.

- Gradient for W1op = (Y_hat - y) * H1
- Gradient for W2op = (Y_hat - y) * H2
- Gradient for W3op = (Y_hat - y) * H3
- Gradient for W0op (Bias) = Y_hat - y

3. Gradients for H3 Weights

- Gradient for W13 = (Y_hat - y) * W3op * H3 * (1-H3) * X1
- Gradient for W23 = (Y_hat - y) * W3op * H3 * (1-H3) * X2
- Gradient for W33 = (Y_hat - y) * W3op * H3 * (1-H3) * X3
- Gradient for W03 (Bias) = (Y_hat - y) * W3op * H3 * (1-H3)

Therefore, using BCE with sigmoid makes the gradient calculation simpler because their derivatives cancel out during backpropagation.